We could do more with upgrading the brush UI elements, such as making the timeline circles highlight and dim if they fall in the brush range. But instead let's look more closely at the brush events fired by the brush control.

### 9.3.3 Understanding brush events

Activity on the brush region fires three separate custom events: `"brush"`, `"brushstart"`, and `"brushend"`. You've probably figured them out based on their names, but for clarity, `"brushstart"` is fired when you mousedown on the brush region, `"brush"` is fired continuously as you drag your mouse after `"brushstart"` and before mouseup, and `"brushend"` is fired on mouseup. In most implementations of a brush, it makes sense to wire it up so that whatever function you want applied with user activity only happens on the `"brush"` event. But you may have functions that are more expensive, such as redrawing an entire map or querying a database. In that case you could use `"brushstart"` to cause a visual change in your map (turning elements gray or transparent) and wait until `"brushend"` to run more heavy-duty activity.

### 9.3.4 Redrawing components

Our entire application responsively resizes whenever we change the dimensions of our browser window. But our brush doesn't resize. It was drawn at a particular size when we created it, and runs off the screen if we make the width too small or looks puny if we make the width too large.

It may seem like you need to delete the graphical elements of the brush and recreate it whenever you redraw your other visual elements. Fortunately, D3 components like axis and brush can be recalled by the containing element to resize them. All you need to do is include in your redraw code the code in the following listing, which now includes reference to the axis and brush components we earlier created.

**Listing 9.17 Component redrawing**

```
var timeTickScale = d3.scale.linear()
     .domain([0,1000])
     .rangeRound([10,1])
     .clamp(true);

var bExtent = timeBrush.extent();

timeScale.range([10,rightSize[0] + leftSize[0] - 10]);
timeAxis.scale(timeScale)
  .ticks(d3.time.hours, timeTickScale((rightSize[0] + leftSize[0])));
timeBrush.x(timeScale);

d3.select("#brushAxis").call(timeAxis);
d3.select("#brushG").call(timeBrush.extent(bExtent));

d3.select("#brushG").selectAll("circle.timeline")
  .attr("cx", function(d) {return timeScale(new Date(d.timestamp))});
```

**Gets the current brush extent**

**We need to dynamically determine the number of ticks on our axis based on available screen space.**

**Sets the current range of timeScale to the width of the two SVGs minus margins**

**Updates timeAxis to the current scale and ticks**

**Updates timeBrush as well**

**Updates the position of the timeline circles**

**Calls both from their respective containers, passing the original extent to make the brush refresh**